TIP102 | Intermediate Technical Interview Prep

Intermediate Technical Interview Prep Fall 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: **134071**

Need help? Post on our **class slack channel** or email us at **support@codepath.org**

Getting Started

Learning with AI ✨

▶ Course Info

Course Progress

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

Week 11

Week 12

Week 5: Session 1

OOP & Linked Lists II

In this session, students will learn to apply Python classes and linked lists through practical exercises. They will begin by creating and manipulating instances of a class and then explore the basics of linked lists, focusing on node creation and linkage. These exercises aim to deepen understanding of object-oriented programming and provide foundational skills in managing custom data structures in Python.

> You can find session recordings and more on the resources tab. Session slide decks are available on the overview tab.

📈 Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.

👷 Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
    - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like VSCode, PyCharm, Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

▶ **Note on Expectations**

🔎 Problem Solving Approach

We will approach problems using the six steps in the UMPIRE approach.

UMPIRE: Understand, Match, Plan, Implement, Review, Evaluate.

We'll apply these six steps to the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem
- **Match** identifies common approaches you've seen/used before
- **Plan** a solution step-by-step, and
- **Implement** the solution
- **Review** your solution
- **Evaluate** your solution's time and space complexity and think critically about the advantages and disadvantages of your chosen approach.

Problem Set Version 1

▼ Problem 1: New Horizons

Step 1: Copy the following code into your IDE.

Step 2: Instantiate an instance of the class `Villager`, which represents characters in Animal Crossing. Store the instance in a variable named `apollo`.

- The `Villager` object created should have the name `"Apollo"`, the species `"Eagle"`, and the catchphrase `"pah"`.

```
class Villager:
    def __init__(self, name, species, catchphrase):
        self.name = name
        self.species = species
        self.catchphrase = catchphrase
        self.furniture = []

# Instantiate your villager here
```

Example Usage:

```
print(apollo.name)
print(apollo.species)
print(apollo.catchphrase)
print(apollo.furniture)
```

Example Output:

```
Apollo
Eagle
pah
[]
```

▶ ✨ AI Hint: Intro to Object Oriented Programming

Close Section

▼ Problem 2: Greet Player

Step 1: Using the `Villager` class from Problem 1, add the following `greet_player()` method to your existing code:

```
def greet_player(self, player_name):
    return f"{self.name}: Hey there, {player_name}! How's it going, {self.catchphrase}!"
```

Step 2: Create a second instance of `Villager` in a variable named `bones`.

- The `Villager` object created should have `name` `"Bones"`, `species` `"Dog"`, and `catchphrase` `"yip yip"`.

Step 3: Call the method `greet_player()` with your name and print out `"Bones: Hey there, <your name>! How's it going, yip yip!"`. For example, if your name is Tram, `"Bones: Hey there, Tram! How's it going, yip yip?"` would be printed out to the console.

▶ ✨ AI Hint: Class Methods

Close Section

▼ Problem 3: Update Catchphrase

In Animal Crossing, as players become friends with villagers, the villagers might ask the player to suggest a new catchphrase.

Adding on to your existing code, update `bones` so that his catchphrase is `"ruff it up"` instead of its current value, `"yip yip"`.

Example Usage:

```
print(bones.greet_player("Samia"))
```

Example Output:

```
Bones: Hey there, Samia! How's it going, ruff it up!
```

▶ ✨ **AI Hint: Class Attributes**

Close Section

▼ Problem 4: Set Character

In the previous exercise, we accessed and modified a player's `catchphrase` attribute directly. Instead of allowing users to update their player directly, it is common to create **setter methods** that users can call to update class attributes. This has a few different benefits, including allowing us to validate data before updating our class instance.

Update your `Villager` class with a method `set_catchphrase()` that takes in one parameter `new_catchphrase`.

- If `new_catchphrase` is valid, it should update the villager's `catchphrase` attribute to have value `new_catchphrase` and print `"Catchphrase updated"`.
- Otherwise, it should print out `"Invalid catchphrase"`.

Valid catchphrases are less than 20 characters in length. They must all contain only alphabetic and whitespace characters.

```
class Villager:
    def __init__(self, name, species, catchphrase):
        self.name = name
        self.species = species
                self.catchphrase = catchphrase
        self.furniture = []

        def set_catchphrase(self, new_catchphrase):
                pass
```

Example Usage:

```
alice = Villager("Alice", "Koala", "guvnor")

alice.set_catchphrase("sweet dreams")
print(alice.catchphrase)
```

```
alice.set_catchphrase("#?!")
print(alice.catchphrase)
```

Example Output:

```
Example 1:
Catchphrase Updated!
sweet dreams
Invalid catchphrase
sweet dreams
```

Close Section

▼ Problem 5: Add Furniture

Players and villagers in Animal Crossing can add furniture to their inventory to decorate their house.

Update the `Villager` class with a new method `add_item()` that takes in one parameter, `item_name`.

The method should validate the `item_name`.

- If the item is valid, add `item_name` to the player's `furniture` attribute.
- The method does not need to return any values.

`item_name` is valid if it has one of the following values: `"acoustic guitar"`, `"ironwood kitchenette"`, `"rattan armchair"`, `"kotatsu"`, or `"cacao tree"`.

```
class Villager:
    # ... methods from previous problems

    # New method
    def add_item(self, item_name):
        pass
```

Example Usage:

```
alice = Villager("Alice", "Koala", "guvnor")
print(alice.furniture)

alice.add_item("acoustic guitar")
print(alice.furniture)

alice.add_item("cacao tree")
print(alice.furniture)

alice.add_item("nintendo switch")
print(alice.furniture)
```

Example Output:

```
[]
["acoustic guitar"]
["acoustic guitar", "cacao tree"]
["acoustic guitar", "cacao tree"]
```

▶ ✨ **AI Hint: Writing Methods**

Close Section

▼ Problem 6: Print Inventory

Update the `Villager` class with a method `print_inventory()` that accepts no parameters except for self.

The method should print the name and quantity of each item in a villager's `furniture` list.

- The name and quantity should be in the format `"item1: quantity, item2: quantity, item3: quantity"` for however many unique items exist in the villager's furniture list
- If the player has no items, the function should print `"Inventory empty"`.

```
class Villager():
    # ... methods from previous problems

    def print_inventory(self):
        # Implement the method here
        pass
```

Example Usage:

```
alice = Villager("Alice", "Koala", "guvnor")

alice.print_inventory()

alice.furniture = ["acoustic guitar", "ironwood kitchenette", "kotatsu", "kotatsu"]
alice.print_inventory()
```

Example Output:

```
Inventory empty
acoustic guitar: 1, ironwood kitchenette: 1, kotatsu: 2
```

Close Section

▼ Problem 7: Group by Personality

The `Villager` class has been updated below to include the new string attribute `personality` representing the character's personality type.

Outside of the `Villager` class, write a *function* `of_personality_type()`. Given a list of `Villager` instances `townies` and a string `personality_type` as parameters, return a list containing the *names* of all villagers in `townies` with `personality` `personality_type`. Return the names in any order.

```
class Villager:
    def __init__(self, name, species, personality, catchphrase):
        self.name = name
        self.species = species
        self.personality = personality
        self.catchphrase = catchphrase
        self.furniture = []
    # ... methods from previous problems

def of_personality_type(townies, personality_type):
    pass
```

Example Usage:

```
isabelle = Villager("Isabelle", "Dog", "Normal", "what's up?")
bob = Villager("Bob", "Cat", "Lazy", "pthhhpth")
stitches = Villager("Stitches", "Cub", "Lazy", "stuffin'")

print(of_personality_type([isabelle, bob, stitches], "Lazy"))
print(of_personality_type([isabelle, bob, stitches], "Cranky"))
```

Example Output:

```
["Bob", "Stitches"]
[]
```

Close Section

▼ Problem 8: Telephone

The `Villager` constructor has been updated to include an additional attribute `neighbor`. A villager's `neighbor` is another `Villager` instance and represents their closest neighbor. By default, a `Villager`'s neighbor is set to `None`.

Given two `Villager` instances `start_villager` and `target_villager`, write a function `message_received()` that returns `True` if you can pass a message from the `start_villager` to the `target_villager` through a series of neighbors and `False` otherwise.

```
class Villager:
    def __init__(self, name, species, personality, catchphrase, neighbor=None):
        self.name = name
```

```
        self.species = species
        self.personality = personality
        self.catchphrase = catchphrase
        self.furniture = []
        self.neighbor = neighbor
    # ... methods from previous problems

def message_received(start_villager, target_villager):
    pass
```

```
isabelle = Villager("Isabelle", "Dog", "Normal", "what's up?")
tom_nook = Villager("Tom Nook", "Raccoon", "Cranky", "yes, yes")
kk_slider = Villager("K.K. Slider", "Dog", "Lazy", "dig it")
isabelle.neighbor = tom_nook
tom_nook.neighbor = kk_slider

print(message_received(isabelle, kk_slider))
print(message_received(kk_slider, isabelle))
```

Example Output:

```
True
Example 1 Explanation: Isabelle can pass a message to her neighbor, Tom Nook. Tom Nook can
message to his neighbor, KK Slider. KK Slider is the target, therefore the function should


False
Example 2 Explanation: KK Slider doesn't have a neighbor, so you cannot pass a message to
```

Close Section

▼ Problem 9: Nook's Cranny

A **linked list** is a new data type that, similar to a normal list or array, allows us to store pieces of data sequentially. The difference between a linked list and a normal list lies in how each element is stored in a computer's memory.

In a normal list, individual elements of the list are stored in adjacent memory locations according to the order they appear in the list. If we know where the first element of the list is stored, it's really easy to find any other element in the list.

In a linked list, the individual elements called **nodes** are not stored in sequential memory locations. Each node may be stored in an unrelated memory location. To connect nodes together into a sequential list, each node stores a reference or pointer to the next node in the list.

Using the provided `Node` class below, create a linked list `Tom Nook -> Tommy` where the instance `tom_nook` has value `"Tom Nook"` which points to the instance `tommy` that has value `"Tommy"`.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

Example Usage:

```
tom_nook = Node("Tom Nook")
tommy = Node("Tommy")
tom_nook.next = tommy
print(tom_nook.value)
print(tom_nook.next.value)
print(tommy.value)
print(tommy.next)
```

Example Output:

```
Tom Nook
Tommy
Tommy
None
```

▶ ✨ AI Hint: Linked Lists

Close Section

▼ Problem 10: Timmy and Tommy

In a linked list, pointers can be redirected to any place in the list.

Using the linked list from Problem 9, create a new Node `timmy` with value `"Timmy"` and place it between `tom_nook` and `tommy` so the new linked list is `tom_nook -> timmy -> tommy`.

Example Usage:

```
print(tom_nook.value)
print(tom_nook.next.value)
print(timmy.value)
print(timmy.next.value)
print(tommy.value)
print(tommy.next)
```

Example Output:

```
Tom Nook
Timmy
Timmy
Tommy
```

```
Tommy
None
```

Close Section

▼ Problem 11: Saharah

Using the linked list from Problem 10, remove the `tom_nook` node and add in a node `saharah` with value `"Saharah"` to the end of the list so that the resulting list is `timmy -> tommy -> saharah`.

```
class Node:
        def __init__(self, value, next=None):
                self.value = value
                self.next = next
```

Example Usage

```
print(tom_nook.next)
print(timmy.value)
print(timmy.next.value)
print(tommy.value)
print(tommy.next.value)
print(saharah.value)
print(saharah.next)
```

Example Output:

```
None
Timmy
Tommy
Tommy
Saharah
Saharah
None
```

Close Section

▼ Problem 12: Print Players Linked List

Write a function `print_list()` that takes in the head of a linked list and returns a string linking together the **values** of the list with the separator `"->"`.

Note: The "head" of a linked list is the first element in the linked list. Equivalent to `lst[0]` of a normal list.

Example Usage:

```
isabelle = Node("Isabelle")
saharah = Node("Saharah")
cj = Node("C.J.")

isabelle.next = saharah
saharah.next = cj

print(print_list(isabelle))
```

Example Output:

```
Isabelle -> Saharah -> C.J.
```

▶ 💡 **Hint: Linked List Traversal**

Close Section

Problem Set Version 2

- ▶ Problem 1: Instantiate an Instance of Player
- ▶ Problem 2: Get Player
- ▶ Problem 3: Update Kart
- ▶ Problem 4: Set Character
- ▶ Problem 5: Add Special Item
- ▶ Problem 6: Print Inventory
- ▶ Problem 7: Race Results
- ▶ Problem 8: Get Rank
- ▶ Problem 9: Tom and Jerry
- ▶ Problem 10: Chase List
- ▶ Problem 11: Update Chase
- ▶ Problem 12: Chase String